

Vue中computed计算属性的核心概念、用法、对比及注意事项

一、computed 计算属性核心概念

`computed` 是 Vue (Vue2/Vue3) 中用于**声明式推导/计算数据**的核心特性，依赖响应式数据缓存，只有依赖项变化时才重新计算，本质是“带缓存的响应式计算函数”。

核心特点：

1. **缓存性**：依赖项不变时，多次访问计算属性会直接返回缓存结果，不重复执行计算逻辑；
2. **响应式**：依赖项更新时，自动重新计算并触发视图更新；
3. **声明式**：写法更接近“数据”而非“函数”，符合 Vue 数据驱动的设计思想。

二、基本用法 (Vue2 vs Vue3)

1. Vue2 选项式 API 用法

Code block

```
1  new Vue({
2    el: '#app',
3    data() {
4      return {
5        firstName: 'Zhang',
6        lastName: 'San',
7        scores: [80, 90, 95]
8      };
9    },
10   computed: {
11     // 1. 只读计算属性 (最常用)
12     fullName() {
13       return `${this.firstName} ${this.lastName}`;
14     },
15
16     // 2. 可读写计算属性 (get/set)
17     fullNameEditable: {
18       get() {
19         return `${this.firstName} ${this.lastName}`;
20       },
```

```

21     set(newValue) {
22         const [first, last] = newValue.split(' ');
23         this.firstName = first || '';
24         this.lastName = last || '';
25     }
26 },
27
28 // 3. 依赖多个数据的计算
29 averageScore() {
30     return this.scores.reduce((sum, score) => sum + score, 0) /
    this.scores.length;
31 }
32 }
33 });

```

模板中使用：

Code block

```

1 <div>{{ fullName }}</div>
2 <div>{{ averageScore }}</div>
3 <!-- 可写计算属性赋值 -->
4 <input v-model="fullNameEditable" />

```

2. Vue3 组合式 API 用法

Vue3 中通过 `computed` 函数创建，返回一个只读的响应式引用对象（需 `.value` 访问）。

Code block

```

1 import { ref, computed } from 'vue';
2
3 export default {
4     setup() {
5         const firstName = ref('Zhang');
6         const lastName = ref('San');
7         const scores = ref([80, 90, 95]);
8
9         // 1. 只读计算属性
10        const fullName = computed(() => {
11            return `${firstName.value} ${lastName.value}`;
12        });
13
14        // 2. 可读写计算属性
15        const fullNameEditable = computed({
16            get() {

```

```
17     return `${firstName.value} ${lastName.value}`;
18   },
19   set(newValue) {
20     const [first, last] = newValue.split(' ');
21     firstName.value = first || '';
22     lastName.value = last || '';
23   }
24 });
25
26 // 3. 依赖多个响应式数据
27 const averageScore = computed(() => {
28   return scores.value.reduce((sum, score) => sum + score, 0) /
29   scores.value.length;
30 });
31
32 return { fullName, fullNameEditable, averageScore };
33 };
```

模板中使用（无需 `.value`，Vue 自动解包）：

Code block

```
1 <div>{{ fullName }}</div>
2 <input v-model="fullNameEditable" />
```

三、computed 与其他方案的对比

1. computed vs methods（方法）

特性	computed 计算属性	methods 方法
缓存性	依赖项不变时，多次访问返回缓存	每次调用都会重新执行函数
调用方式	模板中作为“属性”使用（无括号）	模板中作为“函数”调用（需括号）
响应式触发	依赖项更新时自动重新计算	每次触发重新渲染都会执行
适用场景	基于现有数据的推导（无副作用）	处理业务逻辑、有副作用的操作

示例对比：

Code block

```
1  // Vue2 示例
2  data() {
3    return { a: 1, b: 2 };
4  },
5  computed: {
6    sum() {
7      console.log('computed 执行');
8      return this.a + this.b;
9    }
10 },
11 methods: {
12   getSum() {
13     console.log('methods 执行');
14     return this.a + this.b;
15   }
16 }
```

模板中使用：

Code block

```
1  <!-- 多次访问 sum，仅首次打印「computed 执行」，依赖项不变时复用缓存 -->
2  <div>{{ sum }}</div>
3  <div>{{ sum }}</div>
4
5  <!-- 多次调用 getSum，每次都打印「methods 执行」 -->
6  <div>{{ getSum() }}</div>
7  <div>{{ getSum() }}</div>
```

2. computed vs watch（侦听器）

特性	computed 计算属性	watch 侦听器
核心逻辑	声明式推导数据（从数据到数据）	命令式响应数据变化（执行副作用）
返回值	有返回值（可直接在模板使用）	无返回值（仅执行回调）
触发时机	依赖项更新时自动计算	监听的数据源更新时执行回调

适用场景	数据推导、格式化、过滤	异步操作、复杂业务逻辑（如请求）
------	-------------	------------------

示例对比：

Code block

```
1  // 场景：根据输入的价格（元）计算美元价格（汇率 7.2）
2  data() {
3    return { priceCny: 100, exchangeRate: 7.2 };
4  },
5
6  // computed 实现（推荐，声明式）
7  computed: {
8    priceUsd() {
9      return (this.priceCny / this.exchangeRate).toFixed(2);
10   }
11 },
12
13 // watch 实现（冗余，仅演示）
14 watch: {
15   priceCny: {
16     handler(newVal) {
17       this.priceUsd = (newVal / this.exchangeRate).toFixed(2);
18     },
19     immediate: true
20   }
21 },
22 data() {
23   return { priceCny: 100, exchangeRate: 7.2, priceUsd: '' };
24 }
```

3. computed vs ref/reactive（原始响应式数据）

特性	computed 计算属性	ref/reactive
本质	派生响应式数据（依赖其他数据）	原始响应式数据（独立数据源）
可修改性	只读（除非配置 set）	可直接修改（.value 或赋值）
缓存	有缓存	无缓存（赋值即更新）
适用场景	数据推导	存储原始业务数据

四、watch 与 watchEffect 监听的使用

1. 核心概念区分

在 Vue3 中，监听响应式数据变化的方案主要有 `watch` 和 `watchEffect`，二者均用于响应数据变化并执行副作用，但设计理念和场景存在明显差异：

- **watch：精准监听**指定的响应式数据源，惰性执行（默认首次不执行，需配置 `immediate: true` 才首次执行），能获取数据源的新旧值，属于“命令式”监听。
- **watchEffect：自动追踪**回调函数中使用的所有响应式数据源，非惰性执行（首次会立即执行以收集依赖），无法直接获取新旧值，属于“声明式”监听。

2. 基本用法（Vue3 组合式 API）

2.1 watch 用法

`watch` 支持监听单个数据源、多个数据源，以及复杂响应式对象的属性，需通过 `import { watch } from 'vue'` 引入。

Code block

```
1  import { ref, reactive, watch } from 'vue';
2
3  export default {
4    setup() {
5      // 基础类型响应式数据
6      const num = ref(10);
7      // 引用类型响应式数据
8      const user = reactive({ name: 'Zhang', age: 20 });
9
10     // ① 监听单个 ref 数据源
11     watch(num, (newVal, oldVal) => {
12       console.log('num 变化: ', '新值', newVal, '旧值', oldVal);
13     }, {
14       immediate: true, // 首次执行（默认false）
15       deep: false // 基础类型无需深度监听（默认false）
16     });
17
18     // ② 监听 reactive 对象的单个属性（需用函数返回值形式）
19     watch(() => user.age, (newAge, oldAge) => {
20       console.log('user.age 变化: ', newAge, oldAge);
21     });
22
23     // ③ 监听 reactive 对象（自动深度监听，无需配置 deep: true）
24     watch(user, (newUser, oldUser) => {
25       console.log('user 变化: ', newUser, oldUser);
```

```

26      // 注意：reactive 对象的新旧值是同一引用，需手动深拷贝才能对比差异
27      }, { immediate: true });
28
29      // ④ 监听多个数据源（数组形式）
30      watch([num, () => user.age], ([newNum, newAge], [oldNum, oldAge]) => {
31          console.log('num 或 user.age 变化: ', newNum, newAge);
32      });
33
34      // 触发变化的操作
35      const changeData = () => {
36          num.value += 5;
37          user.age += 1;
38      };
39
40      return { num, user, changeData };
41  }
42  };

```

2.2 watchEffect 用法

watchEffect 无需指定监听目标，自动收集回调内的响应式依赖，通过 `import { watchEffect } from 'vue'` 引入，返回一个用于停止监听的函数。

Code block

```

1  import { ref, reactive, watchEffect } from 'vue';
2
3  export default {
4      setup() {
5          const count = ref(0);
6          const product = reactive({ price: 100, quantity: 2 });
7
8          // ① 基础用法：自动追踪 count 和 product.price
9          const stopWatch = watchEffect(() => {
10              const total = count.value * product.price;
11              console.log('计算结果: ', total);
12              // 副作用操作（如DOM修改、接口请求）
13              document.title = `当前总数: ${total}`;
14          });
15
16          // ② 停止监听（如组件卸载前）
17          const stop = () => {
18              stopWatch();
19          };
20
21          // 触发变化：修改任意依赖项都会执行回调
22          const updateData = () => {

```

```
23     count.value += 1;
24     // product.quantity 未在回调中使用, 修改不会触发监听
25     // product.price 修改会触发监听
26 };
27
28     return { count, product, updateData, stop };
29 }
30 };
```

3. watch 与 watchEffect 核心对比

特性	watch	watchEffect
监听方式	手动指定数据源	自动追踪回调内依赖
执行时机	惰性（默认首次不执行）	非惰性（首次立即执行）
新旧值获取	支持（回调参数直接返回）	不支持（仅能获取当前值）
深度监听	需配置 deep: true（ref数组/对象）	自动深度监听依赖项
停止监听	返回停止函数	返回停止函数
适用场景	需精准控制监听目标、获取新旧值	简单副作用、依赖项较多且无需新旧值

4. 特殊场景处理

4.1 watch 监听 ref 数组/对象

当监听的 ref 数据源是数组或对象时，由于其本质是引用类型，仅修改内部属性不会触发 watch 回调，需配置 `deep: true` 开启深度监听：

Code block

```
1  const arr = ref([1, 2, 3]);
2
3  watch(arr, (newArr, oldArr) => {
4    console.log('数组变化: ', newArr);
5  }, { deep: true });
6
7  // 修改数组内部元素, 会触发监听
8  arr.value[0] = 10;
```


4.2 watchEffect 清除副作用

watchEffect 回调中可返回一个清除函数，用于在依赖项变化或监听停止前清除之前的副作用（如清除定时器、取消接口请求）：

Code block

```
1  watchEffect((onInvalidate) => {
2    const timer = setTimeout(() => {
3      console.log('延迟执行');
4    }, 1000);
5
6    // 清除函数：依赖变化或停止监听时执行
7    onInvalidate(() => {
8      clearTimeout(timer);
9    });
10 });
```

4.3 Vue2 中的 watch 用法（回顾）

Vue2 选项式 API 中仅支持 watch，用法与 Vue3 核心逻辑一致，配置项相同：

Code block

```
1  new Vue({
2    el: '#app',
3    data() {
4      return { num: 10, user: { age: 20 } };
5    },
6    watch: {
7      // 监听基础类型
8      num(newVal, oldVal) {
9        console.log('num 变化', newVal, oldVal);
10     },
11     // 监听对象属性（深度监听）
12     'user.age': {
13       handler(newAge) {
14         console.log('user.age 变化', newAge);
15       },
16       immediate: true
17     },
18     // 监听整个对象（深度监听）
19     user: {
20       handler(newUser) {
21         console.log('user 变化', newUser);
22       },
23       deep: true
```

```
24     }  
25   }  
26 });
```

5. 使用建议

- 若需要**精准监听特定数据**（如仅监听对象的某个属性）、**获取新旧值对比**，优先使用 `watch`；
- 若需要执行**简单副作用**（如根据多个依赖项更新DOM、发送请求），且无需新旧值，优先使用 `watchEffect`，代码更简洁；
- 监听 reactive 对象时，`watch` 需注意新旧值引用相同的问题，如需对比差异需手动深拷贝（如使用 `JSON.parse(JSON.stringify()))`）；
- 组件卸载时，无需手动停止 `watch` 和 `watchEffect`，Vue 会自动清理，但手动停止监听可用于优化性能（如条件性监听）。

五、computed 高级用法与注意事项

1. 避免副作用

computed 应仅做**纯数据推导**，禁止包含异步操作、DOM 修改、状态修改等副作用（这些逻辑应放在 methods/watch 中）。

Code block

```
1  // 错误示例: computed 中包含异步副作用  
2  computed: {  
3    asyncData() {  
4      // 禁止! computed 不能是异步函数  
5      fetch('/api/data').then(res => res.json()).then(data => {  
6        return data;  
7      });  
8    }  
9  }
```

2. 依赖项的注意事项

- computed 仅追踪**响应式依赖**（data/ref/reactive 声明的属性），非响应式数据不会触发重新计算；
- 避免在 computed 中使用 `Date.now()`、`Math.random()` 等非响应式值，否则计算结果不会更新。

3. Vue3 中计算属性的解包

- 模板中自动解包：`{{ fullName }}` 无需 `.value`；
- 脚本中需手动解包：`console.log(fullName.value)`；

- 嵌套在 reactive 对象中自动解包：

Code block

```
1  const obj = reactive({ fullName });
2  console.log(obj.fullName); // 无需 .value
```

五、总结

方案	核心优势	适用场景
computed	缓存、声明式、简洁	数据推导、格式化、过滤（无副作用）
methods	灵活、支持副作用	业务逻辑、事件处理、按需执行
watch	响应变化、支持异步/副作用	数据变化后的复杂操作（如请求）

核心建议：

- 只要是“基于现有响应式数据推导新数据”，优先用 `computed` ；
- 需要“执行操作/副作用”，用 `methods` 或 `watch` ；
- 避免滥用 `watch` 实现数据推导，也避免用 `methods` 替代 `computed` 导致性能浪费。

（注：文档部分内容可能由 AI 生成）